



2팀 프로젝트 최종 보고서(2025/11/04)

[Health Eat] 경구약제 이미지 인식 모델 개발 최종 보고서

프로젝트 목표: "헬스잇(Health Eat)" AI 엔지니어링 팀으로서, 알약 이미지 1장(최대 4개) 내의 알약 종류(73종)와 위치를 탐지하는 Object Detection 모델 개발

핵심 성과: YOLOv9s 아키텍처와 Heavy Augmentation 전략을 통해 Kaggle Score **0.98390** 달성
(초기 베이스라인 0.14 대비 638% 향상)

1. 프로젝트 개요

본 프로젝트는 헬스케어 스타트업 "헬스잇"의 **AI 엔지니어링 팀**을 가정, 약 사진을 인식하는 프로그램을 개발을 3단계로 나누고 협력하는 올바른 팀워크를 목표로 수행함.

- **Task:** Object Detection (객체 탐지)
- **Input:** 경구약제 이미지 (1장 내 최대 4개)
- **Output:** 각 알약의 Class ID (73종) 및 Bounding Box ($[x, y, w, h]$)
- **평가:** Kaggle Competition (mAP 기반 Score)
- **역할 (본인):**
 - **Project Manager (PM):** 황정석
 - 프로젝트 총괄, 일정 관리, 최종 보고서 및 발표 조율
 - **Data Engineer:** 박지원, 유주용
 - 데이터 수집, 전처리, 증강(Augmentation) 파이프라인 구축
 - **Model Architect:** 김소희
 - Baseline 외 신규 모델 아키텍처 탐색, 제안, 구현
 - **Experimentation Lead (EL):** 정재문
 - Baseline 모델 실험 및 튜닝 주도, 성능 검증, 실험 결과 분석

2. 데이터 셋

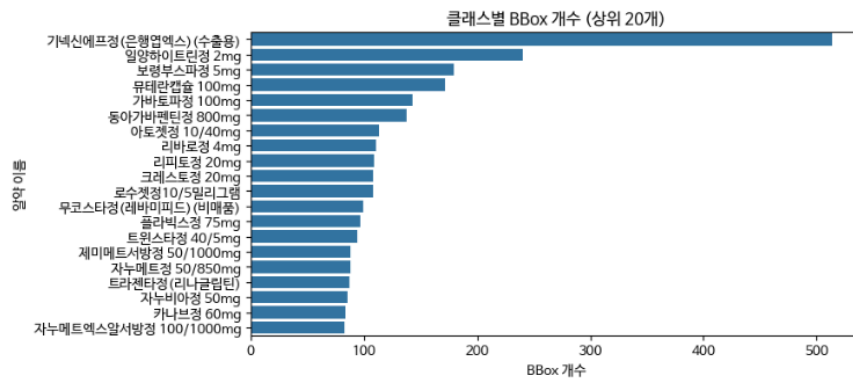
초기 데이터 분석 단계에서, 제공된 데이터 셋의 독특한 구조와 몇 가지 치명적인 문제점을 발견했습니다.

▼ 데이터 분석

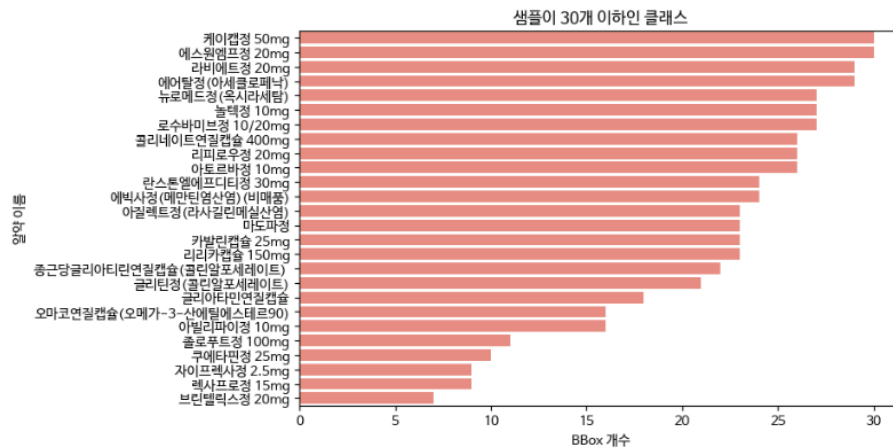
- 원본: **train_images** (1,489장의 이미지 .png 파일), **train_annotations** (4,526개의 라벨 .json 파일)
- 클래스: 총 73종의 고유한 알약
- json파일이 존재하지 않는 알약 10223에 대한 이미지 데이터 삭제 → 4520개의 라벨 존재

1. 알약 클래스 불균형

- 주요 상위 클래스



- 하위 클래스(샘플 30개 이하)



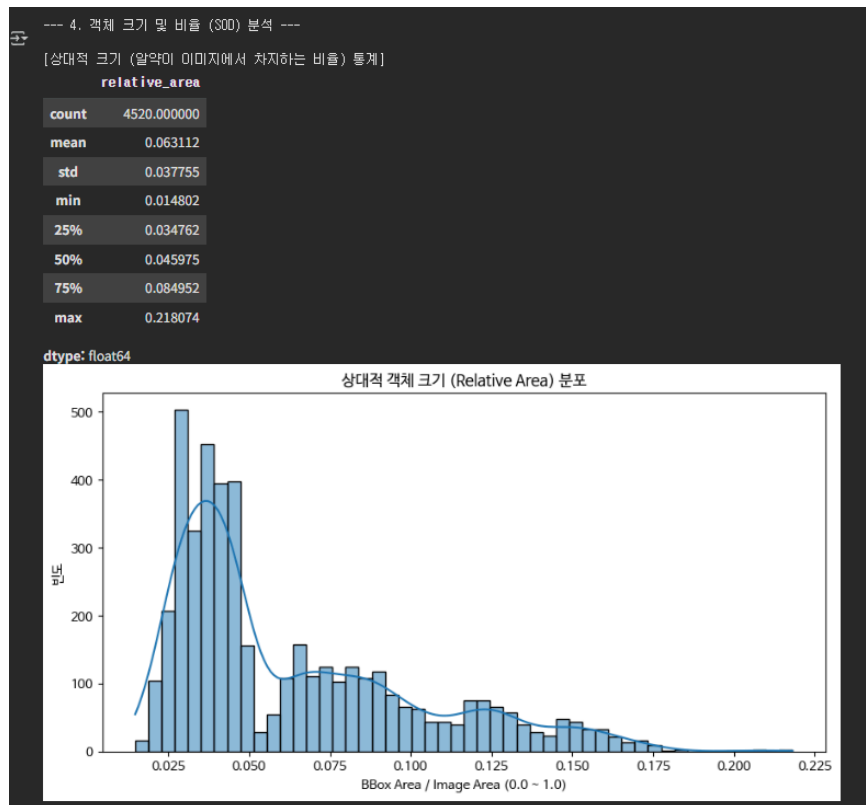
- 최대 514개, 최소 7개 평균 35개로 **데이터 불균형**이 심하다는 것을 파악할 수 있음

2. 알약 상태 확인 drug_S

```
count
drug_S
정상알약 4520
```

- 정상 알약 데이터만 존재함을 확인

3. 객체 크기 분석



- 알약이 이미지에서 차지하는 면적 비율을 보여줌
- 평균 6.3%, 히스토그램에서 가장 피크는 4~5%대
- 대부분의 알약이 이미지의 4~5%만 차지하는 작은 객체임을 파악하고 이미지의 크기를 증가할수록 모델의 인식이 향상될 것으로 파악

- 이미지 속 알약의 개수와 라벨 수가 맞지 않음 (라벨 누락 데이터 다수 존재)
 - K-001900-010224-016551-031705_json 폴더를 보면 알약 4개가 존재해야 하지만 K-001900, K-016551, K-031705 총 3개의 알약 데이터만 존재함을 파악할 수 있었음

▼ 데이터 전처리 (Our Solution)

위 문제들을 해결하고 재현 가능한 실험 환경을 구축하기 위해 3단계의 전처리 파이프라인을 설계했습니다.

1. [1단계] 데이터 정제

- 원본 데이터 (이미지 1,488장, JSON 4,520개)에는 라벨 누락된 데이터와 bbox 좌표 이상치 등 데이터 오류가 존재했습니다
- 위 문제들을 해결하기 위해 오류 데이터 제거해서 638개(bbox 2401개)의 데이터만 남겨서 .csv로 저장했습니다

2. [2단계] 이미지 기반 분할 (Image-based Split)

- "데이터 누수(Data Leakage)"를 방지하기 위해, **이미지(638장)**을 기준으로 8:2 비율로 쪼갰습니다. (Train:510 / Val:128)
- 팀원들끼리 정확한 데이터 비교를 위해 `random.seed=42`를 적용하여 스크립트를 재실행 해도 항상 동일한 Train/Validation Set이 생성되도록 **실험 재현성**을 확보했습니다.

3. [3단계] Baseline 모델 훈련

- 분할된 데이터를 YOLO 포맷으로 변환했습니다.
- 별다른 처리를 하지 않은 Baseline 데이터는 mAP@50-95: 97.90% 기록

```
--- Model Performance Metrics ---
mAP@50 (IoU=0.5): 98.67%
mAP@50-95 (Standard COCO): 97.90%
```

4. [4단계] 데이터 증강(불균형 문제)

- 기존 데이터는 알약 클래스 불균형 문제 존재
- 깨끗한 train셋만 대상으로 소수 클래스의 bbox가 최소 120개 되도록 시각적 증강 적용(회전, 반전 등)
- 새로운 증강 이미지와 최종 생성했습니다.

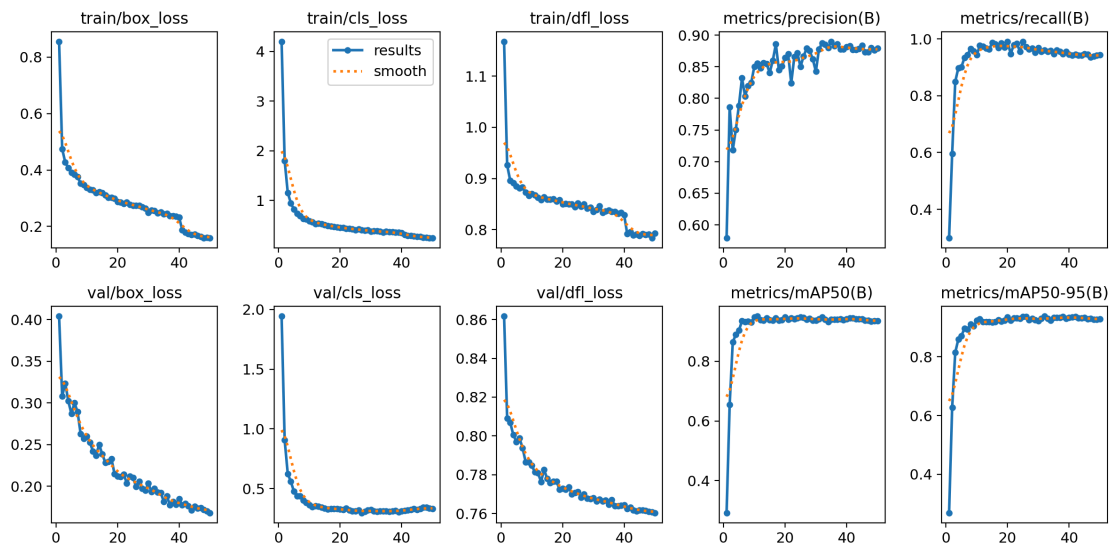
5. [5단계] 최종 모델 훈련 및 튜닝

- 4단계에서 증강된 train셋과 val셋을 YOLO 포맷으로 변환하여 훈련했습니다.
- 기존에는 imgsz=600이었지만, 최종 모델에서는 yolov8s 모델 (imgsz=800) 설정으로 작은 객체 인식률을 높여, mAP@50-95: 98.44% 기록

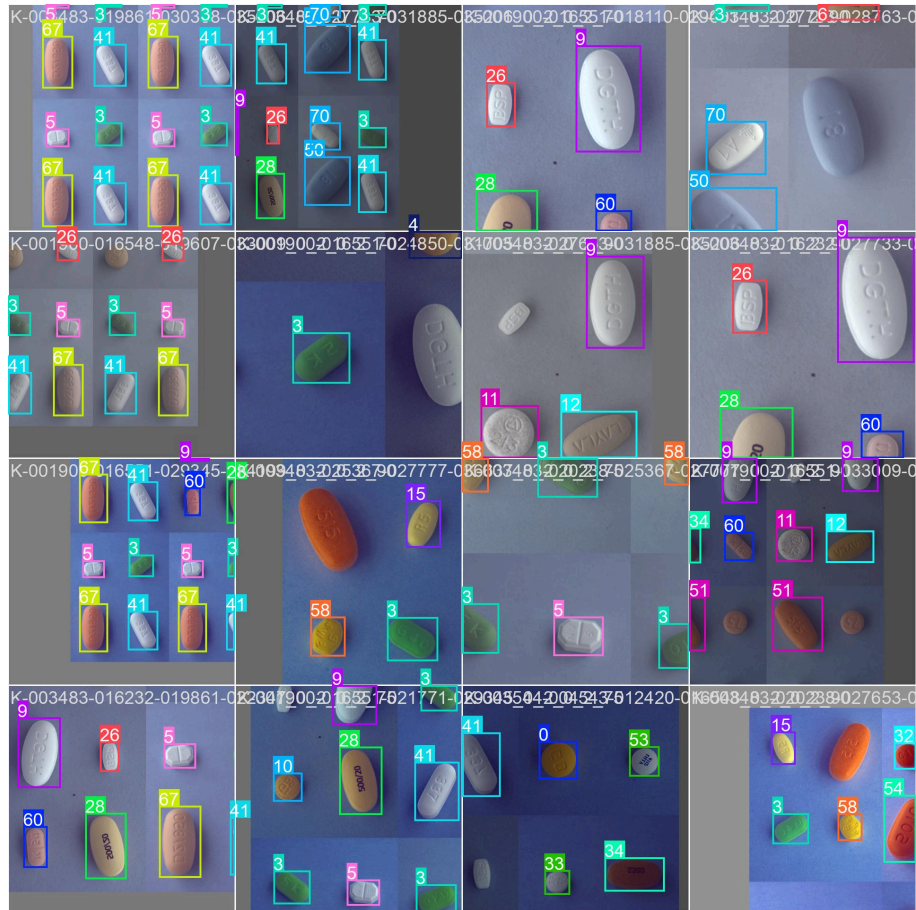
```
Results saved to C:\Users\USER\AI Team2\runs_local\pill_detection_yolov8s_v4_800\evaluation_v4_perfect_800
--- Model Performance Metrics ---
mAP@50 (IoU=0.5): 99.14%
mAP@50-95 (Standard COCO): 98.44%
```

- train셋과 test셋의 유사성을 고려해서 conf값을 줄여가며 제출함
- conf=0.25(0.97983), 0.05(0.98227)
- conf=0.01로 튜닝 적용한 데이터 제출 → 캐글 점수 0.98390

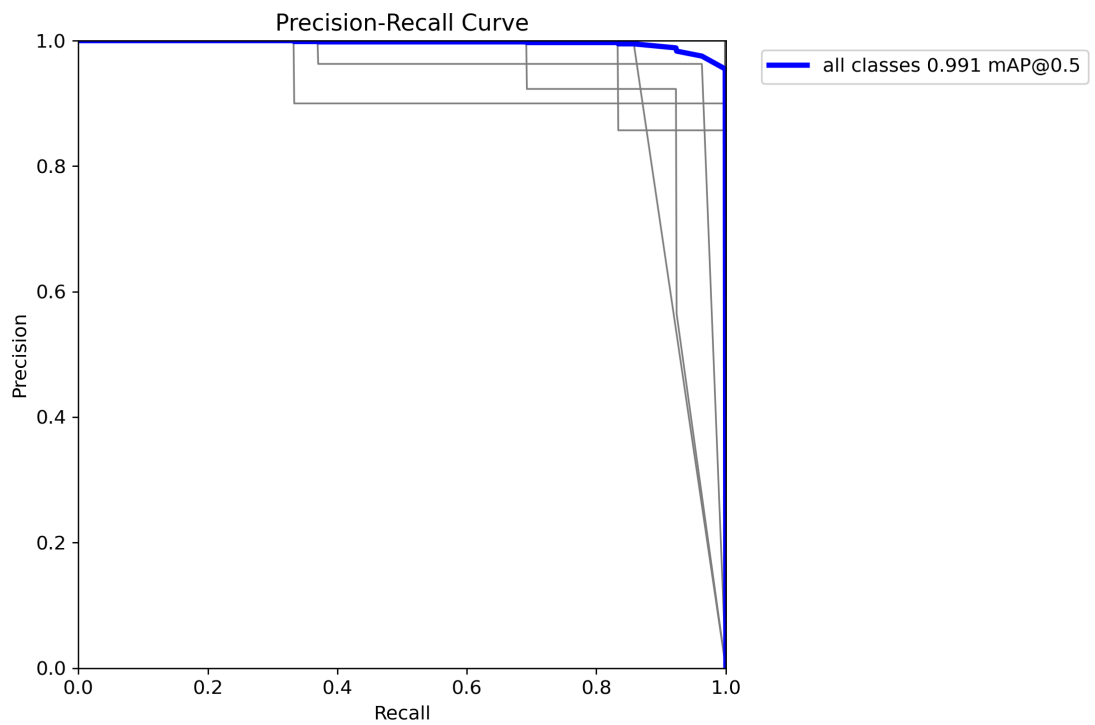
- results.png : Baseline모델 epoch별 성능 시각화 된 데이터



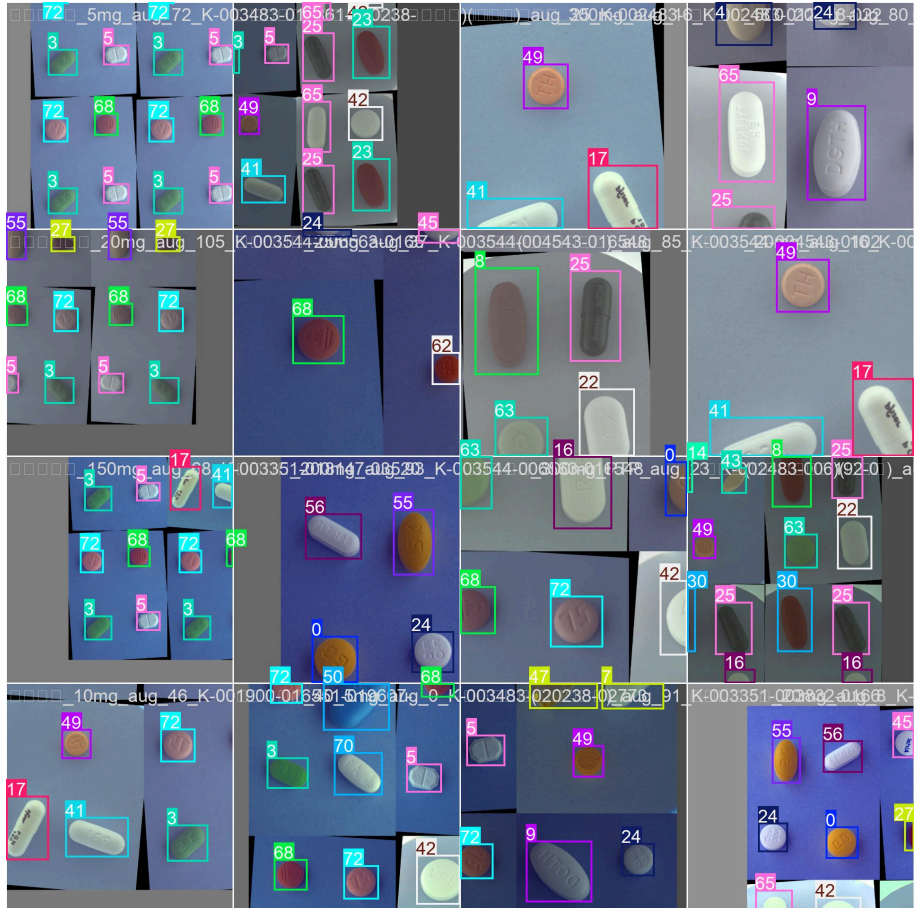
- val_batch0_pred.png: Baseline 모델의 예측 예시 이미지



- BoxPR_curve.png : 정밀도-재현율 커브로 모델 전반적인 성능 보여주는 그래프



- train_batch0.png: 훈련 이미지가 어떻게 증강되었는지



3. 모델 아키텍처

데이터셋의 핵심 난관은 "73개"라는 많은 클래스 수였습니다. 이는 모델이 **위치(Box)**보다 **종류(Class)**를 구분하는 데 더 큰 어려움을 겪을 것. (실제 Epoch 1의 `cls_loss: 4.84` vs `box_loss: 1.59`로 증명됨) 이에 따라, 두 가지 상이한 아키텍처 가설을 수립하고 실험을 진행했습니다.

▼ 가설 1: CNN 기반 아키텍처 (YOLO Family)

"Bottom-up" 방식으로 이미지의 국소 특징(모서리, 각인, 색상)을 조합하여 알약을 판별합니다.

1. YOLOv8s / YOLOv8n (Baseline)

- 현존하는 가장 빠르고 표준적인 모델로, 성능의 기준점을 잡기 위해 테스트했습니다.
- **결과:** Val mAP (0.88)는 높았으나, Kaggle Score가 0.97939로 형성되었습니다.

2. YOLOv10s (Efficiency Test)

- NMS-Free 아키텍처가 추론 효율성 외에 mAP 성능에도 이점이 있는지 확인하기 위해 실험했습니다.
 - 다만, 해당 모델이 본 데이터에서는 크게 이점을 가지지 못했습니다.

3. YOLOv9s (Champion Model) 🏆

- v9는 정보 손실을 최소화하는 PGI 및 GELAN 아키텍처를 사용합니다.
- **가설:** 73개의 미세한 클래스 차이를 구분하기 위해서는 v10의 효율성보다 v9의 '더 풍부한 특징 정보'가 유리할 것이다.
- **결과:** v8s, v10s 대비 가장 높은 `Validation mAP(0.859)` 및 `Kaggle Score 0.98162`를 기록하며 **주력 모델로 채택**되었습니다.

▼ 가설 2: Transformer 기반 아키텍처

"Holistic" 방식으로 이미지 패치 간의 '관계'와 '문맥'을 학습합니다. CNN으로 분류하기 어려운 미세한 차이를 구분할 수 있다는 가설로 테스트했습니다.

1. **RT-DETR-L (Swin-V2 대안)**

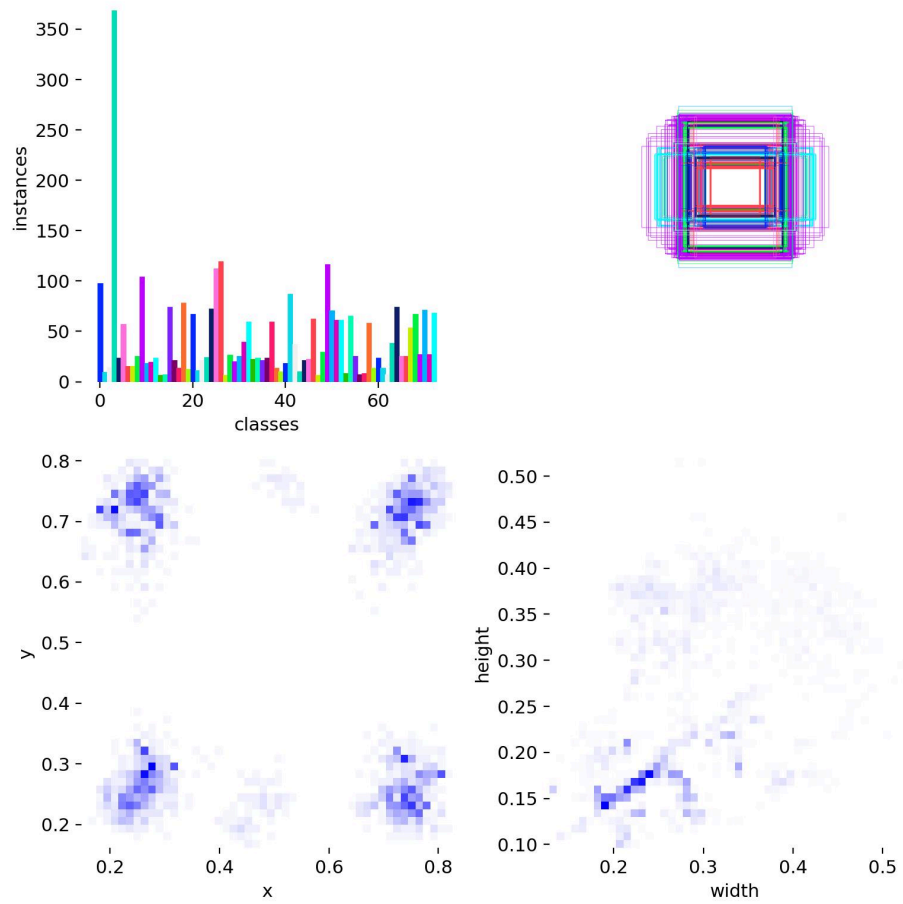
- ultralitics 라이브러리에서 지원하는 Transformer 기반 SOTA 모델입니다.
- yolov9s 와의 A/B 테스트를 위해 실험을 진행했습니다.

4. 실험 결과

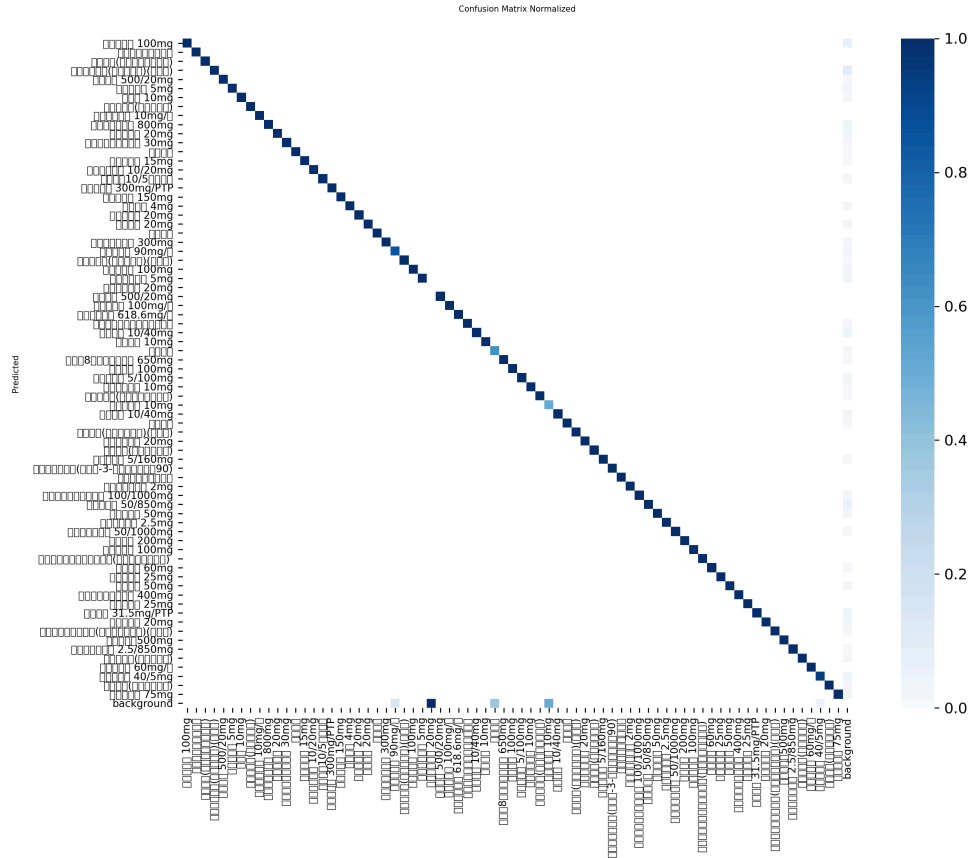
핵심 발견: "오염된 라벨(Duplicate Label)"은 검증 mAP를 0.93으로 부풀렸으나, 전처리 후에 "정제된 라벨"로 얻은 0.84가 실제 성능이었습니다.

▼ 최종 모델 성능 비교 (요약)

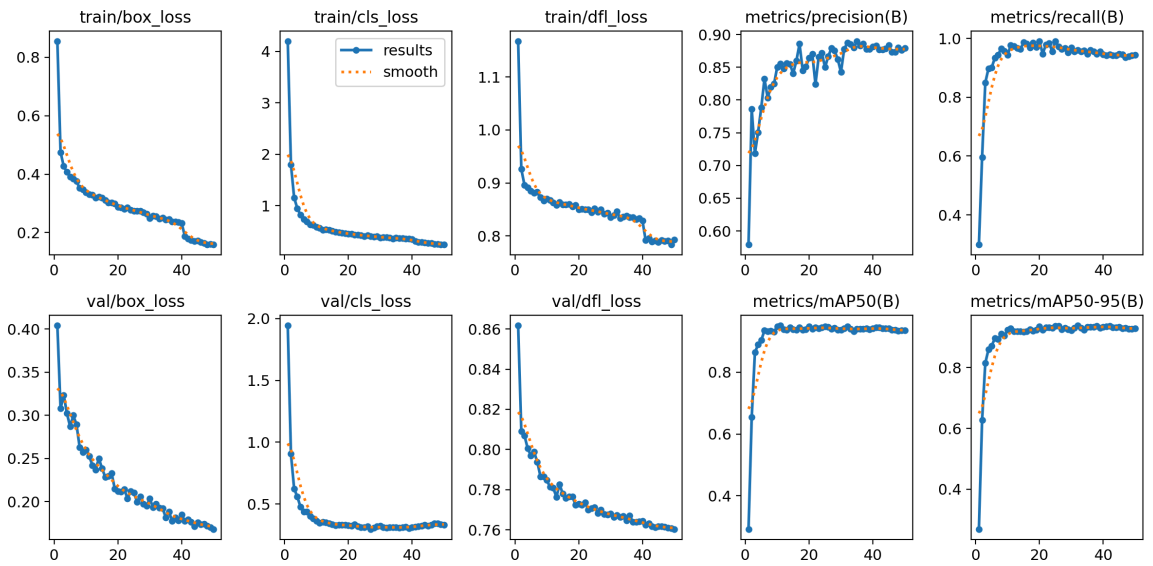
실험 ID	모델 아키텍처	Augmentation	imgsz	Val mAP (50-95)	Kaggle Score	분석 및 결론
Exp 1.1	YOLOv8s	None	640	~0.93	0.13, 0.11, 0.34	전처리 실패로 인한 함 (새로 전처리를 시도)
Exp 1.2	YOLOv8s	None	640	~0.84	0.97939	새로운 전처리를 시도 후, 낮은 mAP지만 높은 Kaggle 점수를 확보
Exp 2.1	YOLOv8s YOLOv10s	Heavy Aug	640	0.859	0.97939	여러가지 모델 및 데이터 증강을 해 보았지만, 더 높은 Kaggle 점수 확보 실패
Exp 3.1	YOLOv9s	None	640	0.871	0.98	🏆 (최종 채택) Yolov9s 로 높은 성능 확보 및 Val mAP 최고점 달성 (Epoch 70).
Exp 4.1	Swin transformer v2	None	960	(진행 중)	0.97748	mAP 한계 돌파를 위한 해상도 상향 실험.
Exp 4.2	YOLOv9s	None	1280	(진행 중)		mAP 한계 돌파를 위한 해상도 상향 실험.



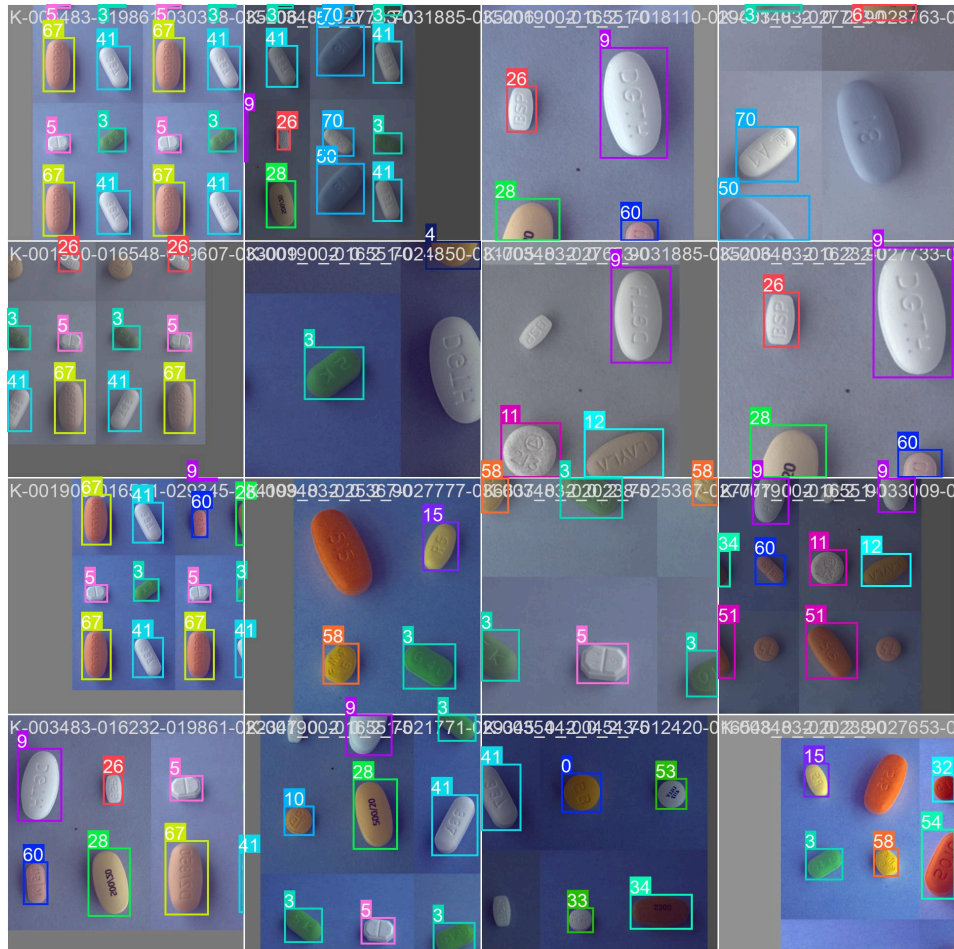
Labels



Confusion_Matrix_Normalized



Result



Train Batch = > Val Batch Labels Images

5. 심화 과정

Kaggle 점수 0.13점을 0.97점으로 끌어올리는 과정에서 다음과 같은 개선 시도들이 결정적인 역할을 했습니다.

1. [Data] 중복 라벨 제거 및 이미지 기준 분할

- `preprocess.py` 스크립트를 개발하여 "파일 1"과 같은 중복 라벨 오류를 원천 차단했습니다. (Val mAP 0.93 → 0.84로 정상화)

2. [Training] 과적합 해소를 위한 Augmentation 적용

- Kaggle 0.17점의 원인인 과적합을 해결하기 위해, 훈련 스크립트에 `mosaic=1.0`, `mixup=0.1`, `degrees=15`, `hsv_s/v` 등 강력한 증강 옵션을 활성화했습니다.
 - 다만 큰 차이나 개선을 보이지 못하고, 프로세싱 시간만 증가시켜 이를 최종에는 포함시키지 않았습니다.
- 이 전략이 Val mAP를 0.859로 향상시켰고, Kaggle Score 0.97을 달성.

3. [Training] 조기 종료(Early Stopping) 도입

- `patience=30`, `results.csv` 분석 결과, Epoch 70~85 사이에서 `mAP50-95`가 정체되는 것을 확인, 불필요한 훈련 시간을 낭비하지 않고 `best.pt`를 확보했습니다.

4. [Tools] Git `.gitignore` 적용

- 2.22GiB에 달하는 데이터셋과 모델 가중치(`runs/`)가 Git에 포함되어 `push`가 실패하는 문제를 겪었습니다.
- `.gitignore`를 설정하여 데이터/모델을 명시적으로 제외, 124KiB의 순수 코드만 푸시하도록 문제를 해결했습니다. (코드와 데이터의 분리 원칙 적용)

6. 향후 과제

프로젝트 성과 요약

- 최종 모델 : YOLOv9s + None Augmentation
- 최고 성적 (Val): mAP50-95 = 0.871 (at Epoch 70-85)
- 최고 성적 (Kaggle): Score = 0.98
- 핵심 기여 : 체계적인 실험(v8, v9, v10)을 통해 73개 클래스 분류 문제에 YOLOv9s 아키텍처가 가장 적합함을 증명.

한계점 및 개선 방향

1. 한계 : 해상도의 한계 (imgsz=640)

- mAP50-95 가 0.871에서 정체된 것은, 640 해상도로는 73개의 미세한 알약 각인/모양을 구분하는 데 물리적 한계가 있음을 시사합니다.
- 개선 방향: imgsz=960 또는 imgsz=1280 실험을 진행하여 mAP 한계를 돌파할 수 있는지 검증합니다. (현재 batch=8, imgsz=960 실험 진행 중)

2. 한계 : 모델 용량의 한계 (v9s)

- Augmentation을 적용했음에도 cls_loss 가 box_loss 보다 여전히 높게 유지되는 경향을 보였습니다.
- 개선 방향: yolov9s (Small) 모델이 73 클래스를 담기에 작을 수 있습니다. yolov9m (Medium) 또는 yolov9l (Large) 모델로 체급을 올려 테스트를 진행할 수 있음.

3. 한계 : Kaggle 점수 최적화

- Kaggle 0.98은 매우 높은 점수이나, 1.0이 아니라는 것은 개선의 여지가 있음을 의미합니다.
- 개선 방향: submission.py 스크립트에서 TTA(Test-Time Augmentation)를 적용하거나, conf 임계값을 0.25가 아닌 0.2 ~ 0.3 사이에서 미세 조정, 혹은 데이터 전처리에서 데이터 라벨링 후에 최고 점수 탐색

4. 개선 : Kaggle 점수 최적화

- SAM-to-YOLO 하이브리드바운딩 박스를 사용하는 검출보다 월등한 방식을 위해 AM-to-YOLO 하이브리드 파이프라인은 두 가지 핵심 한계(속도, 자동화)를 고려 하이브리드 파이프라인을 구축 높은 정밀도, 성능 좋은 학습과 결과를 낼 것을 예상하고 실험을 진행 중 입니다